



Hybrid AI

**Neural and Symbolic
Reasoning**

VL02 · Symbolic foundations

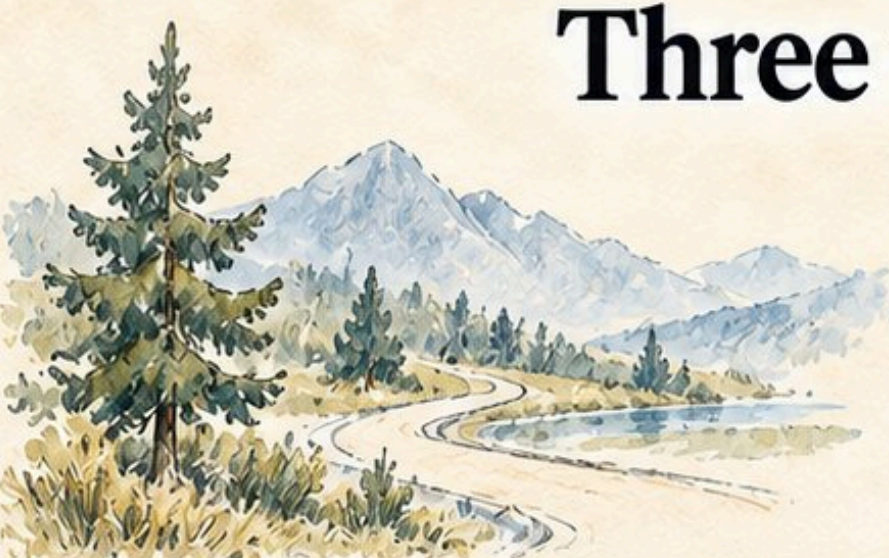
Prof. Dr. Siegfried Handschuh

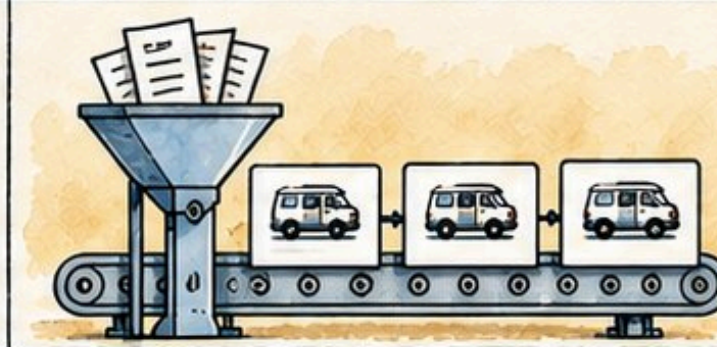
Götz-Henrik Wiegand and Lorena Raichle

22 September 2026 · University of St.Gallen

Three ways to build a campervan search agent

A qualitative assessment for this use case



			
Criterion	LLM agent	Python pipeline	FrameX hybrid AI
Decision quality	★ ★	★ ★ ★	★ ★ ★ ★ ★
Maintainability	★ ★ ★	★ ★ ★	★ ★ ★ ★ ★
Running cost	★ 24\$	★ ★ ★ ★ ★	★ ★ ★ ★ ★ 0.8\$
Initial setup ease — manual	★ ★ ★ ★ ★	★ / ★ ★	★
Initial setup ease — AI-assisted coding	N/A prompt-configured	★ ★ ★ ★	★ ★ ★ ★

Scale: 1–5 stars. More stars = more favourable.

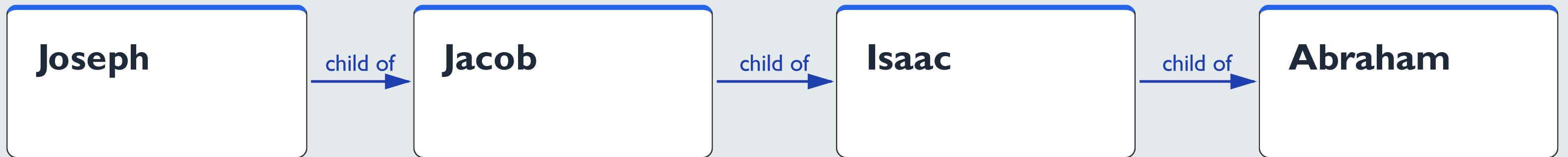
For cost and setup, more stars mean lower cost or less effort.

Python manual setup: beginners / computer scientists.

Illustrative assessment for this use case, not a measured benchmark.

PREDICT

Who descends from whom?



Does Joseph descend from Abraham? Write down every relationship that holds without being written down.

You are here: session 2 of 12

Sessions	Capability	Next use
01 · ► 02 · 03	Represent and derive	H1 · 29 September
04 · 05 · 06 · 07	Graph queries and language	H2 · 10 November
08 · 09 · 10	Planning and adaptation	H3 · 1 December
11 · 12	Learning and architecture choice	Transfer

Today's milestone: you can trace a derivation by hand and say which assumption made a negative answer legitimate.

Today

01

REPRESENT

Facts, variables, joins.

02

DERIVE

Rules, recursion, rounds,
explanations.

03

MODEL

Frames and the right
entity.

04

DECIDE

The wedding, in two
sections: evidence, then
assumptions.

TODAY'S LEARNING GOALS

By the end you can check yourself against these

REPRESENT · Write a fact, read it back, and say what reversing it would claim.

DERIVE · Predict the round in which an indirect ancestor appears, and explain why inference stops.

MODEL · Decide which entity carries which attribute, and why.

DECIDE · For a missing record, name the assumption that turns *unknown* into *false*.

Represent

A fact is a claim with a direction

REPRESENT

A fact names a relationship

Expression	Reading
isaac[father → abraham]	Isaac's father is Abraham
jacob[mother → rebekah]	Jacob's mother is Rebekah
isaac[ancestor → abraham]	Isaac descends from Abraham, directly or through a chain

F-Logic Object Oriented: **Object** on the left, **named property** in the middle, **value** on the right.

Turn it around and you claim something else: `abraham[father → isaac]` .

Linguistic logic: `subject[attribute → object]`

MATCH

A variable holds whatever matched

Query & Bindings

```
?- ?C[father -> ?P].
```

Answers returned:

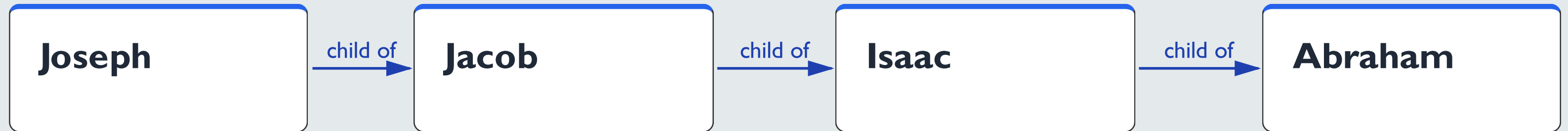
```
C = isaac,    P = abraham  
C = jacob,    P = isaac  
C = joseph,   P = jacob
```

How it works:

- The question mark marks a variable: Anything that begins with a `?` (such as `?C` or `?P`) is a variable and serves as a placeholder.
- The query: The expression `?- ?C[father → ?P].` is a query to the system: "Give me all combinations of child (`?C`) and father (`?P`) that are stored in the database.
- Concrete bindings: The engine matches the query against the known facts and returns concrete pairs of values

COMPUTE

A join is just a shared variable



`?C[father -> ?F] AND ?F[father -> ?GF]`

`C = jacob, F = isaac, GF = abraham`

`C = joseph, F = jacob, GF = isaac`

The same `?F` connects the two.

Mini-Check: why is (joseph, abraham) not a two-hop result here?

Three things to keep

01

DIRECTION

`a[father → b]` and
`b[father → a]`
are different claims.

02

BINDING

A variable stands for every value
that matches, one answer per
binding.

03

JOIN

A shared variable is the whole join.
Nothing else connects two facts.

Derive

From three facts to every ancestor

A rule derives new facts

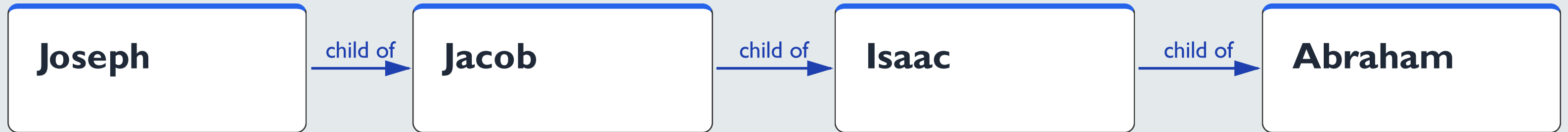
```
?C[ancestor → ?P] ← ?C[father → ?P].
```

- **Imperative** (The "How"): Like a step-by-step cooking recipe. You tell the computer every single action to take using loops, variables, and conditions (e.g., Python, Java).
- **Declarative** (The "What"): Like ordering food at a restaurant. You just state what result you want or what rules apply (e.g., SQL, Datalog), and the system figures out how to calculate it on its own.

- **Logical Implication:** Not an imperative control-flow if, but a declarative deduction (**Head** if **Body**).
- **Fact Factory:** Every variable binding that matches the body generates and adds a new fact to the head.
- **Scope:** This baseline rule covers direct fathers only.

¹ Alfred Horn (1918 to 2001), an American mathematician

One rule handles any chain length



Step 1 · Run again to restart sequence

```

?C[ancestor -> ?P] <- ?C[father -> ?P].
?C[ancestor -> ?A] <- ?C[father -> ?F] AND ?F[ancestor -> ?A].
  
```

Step 2 · Uses previous steps

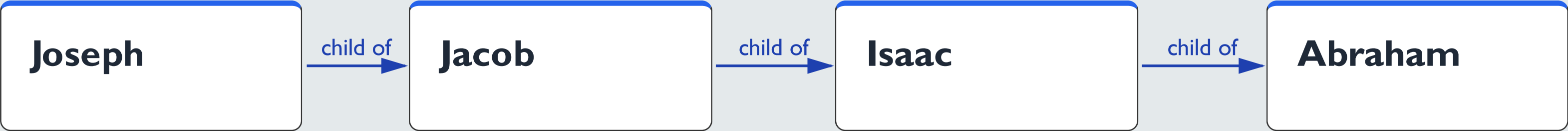
```

?- joseph[ancestor -> abraham].
?- ?C[ancestor -> ?A].
  
```

The second rule puts one direct step in front of an ancestry you already have. Any chain length, no rule per generation.

Run: first «does Joseph descend from Abraham», then every ancestor pair.

Predict two rounds before executing



Round	New ancestor facts
1	(isaac, abraham), (jacob, isaac), (joseph, jacob)
2	Your prediction
3	Your prediction
4	Has the set stopped changing?

Inputs are present before round 1. Facts are retained between rounds.

Each round only sees the round before

Round	New ancestor facts
1	(isaac, abraham), (jacob, isaac), (joseph, jacob)
2	(jacob, abraham), (joseph, isaac)
3	(joseph, abraham)
4	None: fixpoint detected

```
?C[ancestor -> ?P] <- ?C[father -> ?P].  
?C[ancestor -> ?A] <- ?C[father -> ?F] AND ?F[ancestor -> ?A].
```

Inputs are present before round 1.
Facts are retained between rounds.

Stop when no new fact can be added

The Loop (Least Fixpoint)

- **Start:** Begin with initial input facts.
- **Apply:** Run all rules against current knowledge.
- **Accumulate:** Add new facts while keeping old ones.
- **Repeat:** Loop until a round adds **zero new facts** (the stable state).

Why It Must Stop

- **Finite Vocabulary:** Only a fixed set of names/identifiers exist.
- **No New Names:** Rules combine existing items; they never invent new ones.
- **Hard Upper Bound:** With 4 names, max 16 possible pairs can ever exist.
- **Set Semantics:** Deriving a known fact twice adds nothing. It *must* hit a ceiling.

Where did that come from?

joseph[ancestor → abraham]

rests on

joseph[father → jacob] and jacob[ancestor → abraham]

which in the end rest on

jacob[father → isaac] and isaac[father → abraham]

COUNTERFACTUAL CHECK

Take one fact away

Step 1 · Run again to restart sequence

```
isaac[father -> abraham].  
// jacob[father -> isaac] removed  
joseph[father -> jacob].
```

Step 2 · Uses previous steps

```
?- ?C[ancestor -> ?A].
```

Same two rules.

Predict which of the six ancestor pairs survive,
then run.

Mini-Check: would deleting just one displayed conclusion have been enough?

Same pattern, different world

Another approach to this lecture could have connected places and asked which destinations you can reach.

Replace `father` with `edge` and `ancestor` with `reachable`. The **rule pattern** transfers. The **domain meaning** changes.

Does reachability tell us travel time, an itinerary or whether the line runs today?

Four things to keep

01

RULE

Head if body. Every matching binding adds the head.

02

ROUNDS

A round sees only the round before. Chain length n needs n rounds.

03

FIXPOINT

Finite names, no invented terms: the loop must stop.

04

EXPLAIN

Every derived fact has a chain back to inputs. Remove an input, recompute.

Model

Which thing carries which attribute?

What is an F-Logic Frame?

```
Person { label: String, born: Integer }.
```

```
Man extends Person.
```

```
jacob:Man[  
  label -> "Jacob",  
  born -> 1850,  
  father -> isaac,  
  mother -> rebekah  
].
```

- **Unique Identity (Handle):** `jacob` is the fixed ID of the object, not just an interchangeable string.
- **Class Membership:** Through `jacob:Man`, the object belongs to the class Man (and thereby inherits the structure of Person).
- **Multiple Slots (Properties):** The object holds various property slots within brackets (`label` , `born` , `father` , `mother`), each linked to concrete values or other objects.

An entity needs a name that stays

```
world open.  
Person { label: String }.  
Man extends Person.  
isaac:Man[label -> "Isaac"].  
jacob[father -> isaac].
```

Key Takeaway: Handles vs. Labels

- *The Handle* (isaac): The unique, permanent system ID. It never changes and uniquely identifies the object.
- *The Label* ("Isaac"): Just a descriptive text attribute stored inside a slot. Labels can repeat (two different people could theoretically both have the label "Isaac"), but their system handles remain distinct.

A person is not a birth record

```
// Defining the Event Class
Birth { father: Person, mother: Person, child: Person, order: Integer }.

// Creating a Specific Event Instance
birth1 : Birth.
birth1[father -> jacob; mother -> rachel; child -> joseph; order -> 11].
```

- `jacob` identifies the person. `birth1` identifies this specific birth.
- Losing the birth record does not erase the person.
- **Rule of Thumb:** Events get their own entries so that dynamic or multi-part facts (like orders, timestamps, or transactions) never accidentally overwrite core entity data.

Constrain, compare, parameterise

01

DECLARE

```
declare required Person label.  
declare functional father.  
declare inverse father childOf.
```

`framex validate` reports every violation.

02

COMPARE

```
?C:LateBorn ←  
  ?B:Birth[child → ?C; order → ?R]  
  AND ?R > 11.
```

The engine compares numbers it has. It never computes new ones.

03

METHOD

```
?F[childWith(?M) → ?C] ←  
  ?B:Birth[father → ?F;  
            mother → ?M;  
            child → ?C].
```

`jacob[childWith(rachel) → ?C]`

Parameterized Relations

Three things to keep

01

IDENTITY

An identifier stays. A label is just an attribute.

02

CLASS

`isaac:Man` instance and
`Man extends Person` subclass,
i.e., inheritance.

03

EVENT

A record of something that happened is its own entity.

Evidence

Who gets into Jacob's wedding?

My first guest-list rule was wrong

Step 1 · Run again to restart sequence

```
world open.  
wedding[requires -> related].  
wedding[requires -> reply].  
esau[has -> related].
```

Step 2 · Uses previous steps

```
?G[admitted -> wedding] <- ?G[has -> ?R] AND wedding[requires -> ?R].  
  
?- esau[admitted -> wedding].
```

The wedding requires being **related** and a **reply**. Esau is related. Nobody has his reply.

The rule still derives `esau[admitted → wedding]`.

It finds **a** matching requirement. It never checks **all** of them.

Requirements can have two levels

Relatives group

Related to Jacob **OR** related to Rachel

One is enough.

Reply group

A reply has arrived

Required as well.

Admitted = Relatives AND Reply

WORKED EXAMPLE

One guest, all the way through

Evidence for Laban	Status
Related to Jacob	Unknown
Related to Rachel	True
Reply arrived	True

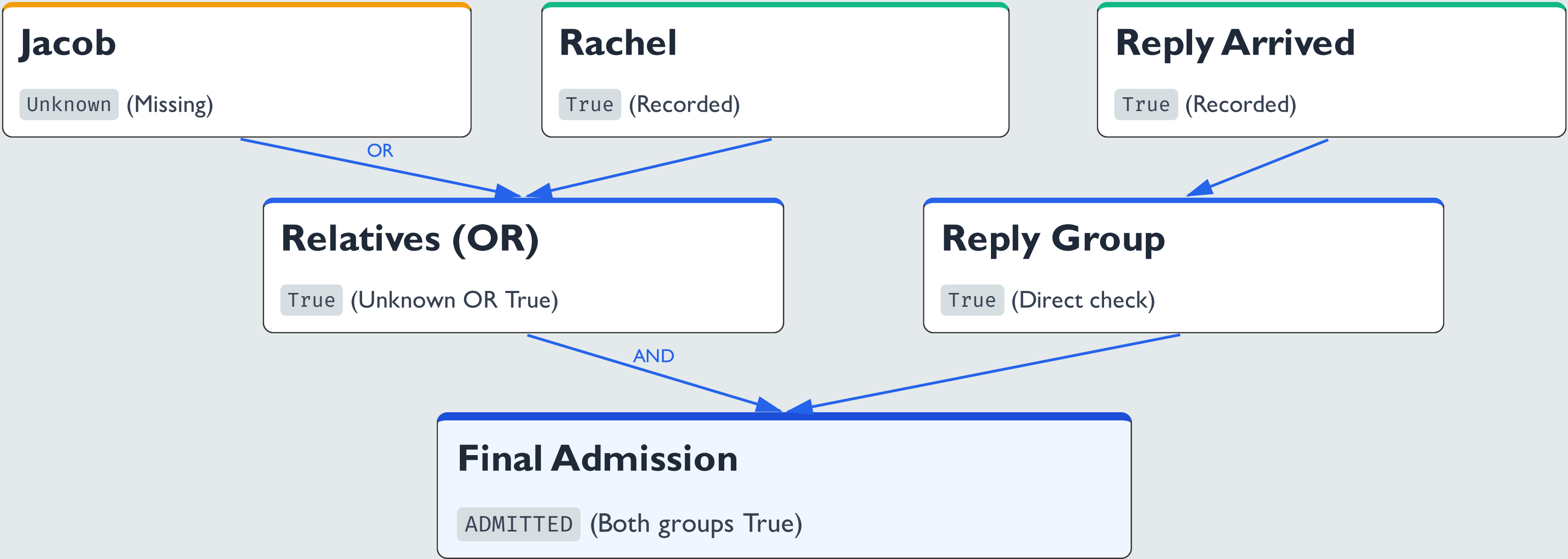
Evidence has three values: **true**, **false** (known not so), **unknown** (nothing on file).

Relatives: **true**, related to Rachel is enough.

Reply: **true**.

Admitted: **true**, both required groups are satisfied.

Tracing Laban's Admission



PREDICT FOUR CASES

Now change only the evidence

Guest	Related to Jacob	Related to Rachel	Reply arrived	Admitted?
Laban	Unknown	True	True	?
Esau	True	Unknown	Unknown	?
Leah	False	True	False	?
A stranger	False	False	True	?

Here **false** means known not so, not merely absent.

How the three values combine

OR Within a Group

Any alternative true → **True**

AND Across Groups

All groups true → **True**

Operation	True when	False when	Otherwise
OR within a group	Any alternative true	All alternatives false	Unknown
AND across groups	All groups true	Any group false	Unknown

A wedding with **no required groups** admits everyone by this model.
An empty required group is malformed input under the HI contract.

Two different questions

Derived Knowledge

Discovered via rules (e.g., blood relations)

Recorded Evidence

Anchored to reality (e.g., replies, payments)

Query	Uses	Does not imply
Ancestry	Father and mother facts	Anyone replied to anything
Admission	Groups plus actual evidence	The guest is related by blood

Key Takeaway

- **Rules/Derivation** let you discover hidden connections.
- **Ground Facts/Evidence** anchor your system to real-world actions.
- **Never mix them up:** A graph of relationships and a table of evidence must be kept distinct!

Three things to keep

01

QUANTIFIER

A body match is *some*, never *all*.
Model groups explicitly.

02

THREE VALUES

One false group sinks an AND.
One true alternative carries an
OR.

03

RECORDED

Relatives can be derived. A reply
can only be recorded.

Assumptions

What a gap in the record means

Missing is not the same as false

Step 1 · Run again to restart sequence

```
world open.  
laban[replied -> wedding].
```

Step 2 · Uses previous steps

```
?- laban[replied -> wedding].  
?- esau[replied -> wedding].
```

Two questions, in order: did Laban reply, did Esau reply.

The list may simply lack the entry.

CLOSED WORLD

A closed world is a completeness claim

Step 1 · Run again to restart sequence

```
world closed.  
laban[replied -> wedding].
```

Step 2 · Uses previous steps

```
?- laban[replied -> wedding].  
?- esau[replied -> wedding].
```

Same two questions, one word changed.

Justified only if the reply list is complete for the question you are asking.

BOUNDED NEGATION

Close only the list that is complete

Step 1 · Run again to restart sequence

```
world open.  
closed class declined.  
esau:invited.  
laban:invited.  
leah:invited.  
laban:declined.  
?X:expected <- ?X:invited AND NOT ?X:declined.
```

Step 2 · Uses previous steps

```
?- laban:declined.  
?- esau:expected.  
?- laban:expected.
```

The declined list is complete. Nothing else is. Predict, then run, in this order: `laban:declined`, `esau:expected`, `laban:expected`.

Three answers, three reasons

laban:declined → True

Explicitly listed in closed class

esau:expected → True

Invited, and NOT declined

laban:expected → Unknown

Underscored, expected is open

Ground claim	Result	Why?
laban:declined	True	Explicitly listed in a class declared complete
esau:expected	True	Invited, and not declined
laban:expected	Unknown	Nothing derived it, and expected is open

Closing declined does not close expected . The rule is an implication, not a definition.

Three things to keep

01

WORLD

Closed world is a claim about your record, declared, never inferred.

02

SCOPE

Close one list. Its closure does not leak into what you derive from it.

03

DECLARE

A *false* is only as good as the sentence that says what you assumed.

The state comes before the sentence

Monotone Rules

Only add facts. They never retract or remove past state.

State Transitions

Required for actions that remove or replace state (VL08 / VL09).

«Esau gives Jacob the key at the wedding»

says something about a changing state:

who has the key?

Rules that only **add facts**

do not implement actions that **remove or replace state**.

That is VL08 and VL09.



A good test splits two plausible programs

Test	What it distinguishes
Three-hop ancestry	Direct lookup vs recursive inference
Only one of two required groups succeeds	Existential shortcut vs AND
One successful alternative; another unknown	OR vs requiring every alternative
A missing reply	Unknown vs assumed false
Related, no reply	Admission vs ancestry

Summary



LOGIC PUZZLE CHALLENGE

Logic Puzzle with FrameX

Three Guests in a Café

Anna, Ben and Clara each sit alone at one of the three tables: Red, Blue and Green. Each orders a different drink: water, wine or juice.

Find out who sits at which table and which drink each one is having.

The clues:

- 1. Water is being drunk at the red table.
- 2. Ben is not sitting at the blue table.
- 3. No wine is being drunk at the green table.
- 4. Clara is sitting at the blue table.
- 5. Anna does not drink juice.

Who drinks what? Which table does Anna sit at?

Step 1 · Run again to restart sequence

```
world closed.

// Only the domains are given as facts.
red:Table.
blue:Table.
green:Table.

water:Drink.
wine:Drink.
juice:Drink.

// Clue 1: The red table serves water.
// Clue 3: The green table does not serve wine.
?Table[allows -> ?Drink] <-
    ?Table:Table AND ?Drink:Drink AND
    (?Table != red OR ?Drink = water) AND
    (?Table != green OR ?Drink != wine).

// Construct complete assignments and retain those satisfying
cafe[solution(?AnnaTable, ?AnnaDrink,
              ?BenTable, ?BenDrink,
              ?ClaraTable, ?ClaraDrink) -> true] <-
    ?AnnaTable[allows -> ?AnnaDrink] AND
    ?BenTable[allows -> ?BenDrink] AND
    ?ClaraTable[allows -> ?ClaraDrink] AND

    // Each guest sits at a different table.
    ?AnnaTable != ?BenTable AND
    ?AnnaTable != ?ClaraTable AND
    ?BenTable != ?ClaraTable AND

    // Each guest drinks a different beverage.
    ?AnnaDrink != ?BenDrink AND
    ?AnnaDrink != ?ClaraDrink AND
    ?BenDrink != ?ClaraDrink AND

    // Clue 2: Ben does not sit at the blue table.
    ?BenTable != blue AND
    // Clue 4: Clara sits at the blue table.
    ?ClaraTable = blue AND
    // Clue 5: Anna does not drink juice.
    ?AnnaDrink != juice.
```

Step 2 · Uses previous steps

```
?- cafe[solution(?AnnaTable, ?AnnaDrink,
                ?BenTable, ?BenDrink,
                ?ClaraTable, ?ClaraDrink) -> true].
```

SUMMARY

Four labels, closed



REPRESENT



You read a fact with its direction.



DERIVE



You predicted the round and explained the stop.



MODEL



You put the attribute on the right entity.



DECIDE



You named the assumption behind a *false*.

Kahoot Quiz:Active Recall

[Kahoot Quiz VL02](#)

play.kahoot.it

Working vocabulary

Term	Meaning here
Binding / substitution	Values assigned to variables in a match
Join	Matching relations through shared variables
Recursion	A derived relationship is used to derive more of it
Fixpoint	Applying the rules adds no new facts
Frame	An entity with typed attributes
Object notation	A fact written as <code>object[property → value]</code>
World assumption	Declared once per program: <code>world open.</code> absence is unknown; <code>world closed.</code> absence is false
Closed class / slot	A class or slot declared complete with <code>closed class</code> or <code>closed slot</code> ; NOT over it reads absence as failure, nothing else is closed by it

Sources

FrameX F-Logic tutorial, `framex-flogic-tutorial.pdf` in the FrameX repository: the family facts, joins, rules and frames used today.